The information in a feed-forward neural network flows only in one way — from the input layer to the output layer, passing through the hidden layers. Feed-forward neural networks have no recollection of the information they receive and are bad at predicting what's coming next. A feed-forward network has no concept of time order because it only analyses the current input. It has no recollection of what happened in the past, apart from its training. An RNN's information is cycled in a loop. It considers the current input as well as what it has learned from previous inputs before making a decision. Recurrent neural networks, to put it simply, blend information from the past with information from the present. This signifies that the output will be sent back to the input and then used when the next input comes along, in the next time step. Essentially, the network's 'state' is propagated forward in time. RNNs are neural networks that travel across time.

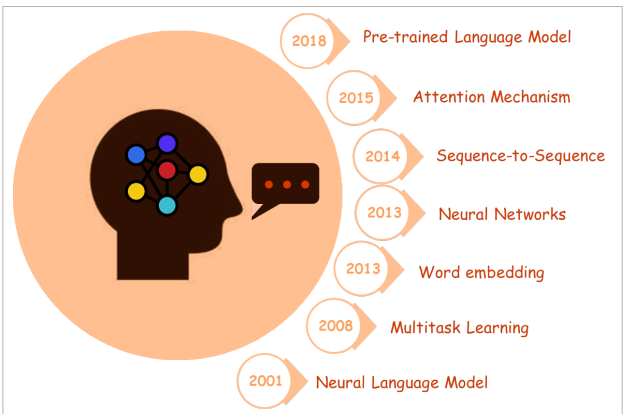

Figure 1: Milestones of NLP technologies

Figure 2 illustrates the difference in information flow between a feed-forward neural network and RNN.

In the feed-forward neural networks, all test cases are considered to be independent. Take an example of sequential data, which can be the flight booking data for an airline company. A simple machine learning model or an artificial neural network may learn to predict ticket demands based on several features — source-destination, schedule-time (morning, midnight), etc. While the demand for tickets depends on these features, it is also largely dependent on the ticket demand in the previous days or weeks, or months. In fact for a trader, these values in the previous days (or the trend) are one major deciding factor for sales predictions. Recurrent neural networks are used to achieve this time dependency. A typical RNN is shown in Figure 3.

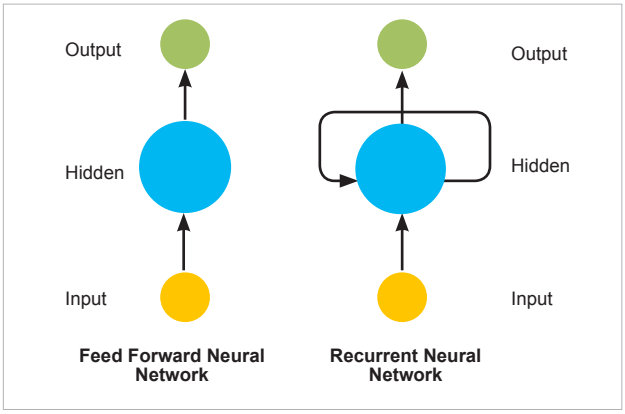

Figure 2: Feed-forward neural network and RNN

RNNs are widely utilised in natural language processing. They have sharing features learned across multiple text positions, which is not the case with a normal neural network. Traditional neural networks are unable to predict future inputs based on previous ones.

LSTM (long short-term memory): An improvement over RNN

RNNs are wonderful for short contexts, but we need our models to be able to grasp and recall the context behind the sequences, just like a human brain, to form a collection and remember it. With a simple RNN, this is not achievable. For example, in the sentence '*The lemon tastes* _____.' RNNs turn out to be quite effective. This is because this problem has nothing to do with the context of the statement. The RNN need not remember what was said before this, or what its meaning was. All it needs to know is that in most cases the lemon is sour. Thus the prediction would be: '*The lemon tastes sour.*'

Another example is, '*I spent many years in my early career in Japan. Then I came back to my motherland. ….. I can speak fluent __?__*. We can understand that since the author has worked in Japan for many years, it is very likely that he may possess a good command of Japanese. However, to produce an accurate forecast, the RNN must remember the context. A RNN fails in this situation! The problem of 'vanishing gradient' is the explanation for this. A large amount of unnecessary data may separate the relevant information from the point where it is required. To understand this, you must first understand how a feed-forward neural network learns. We know that in a conventional feed-forward neural network, the weight updation applied to a specific layer is a multiple of the learning rate, the error term from the previous layer, and the input to that layer. RNN remembers things for just a small period, i.e., if we just need the information for a short period, it may be replicable, but once a large number of words are